Grade Calculator Example in TypeScript + React

How to Build and Run TypeScript + React Code

Compilation of TypeScript files

We need to compile the TypeScript + React source (tsx).

Install Node.js

This is part of HW1: it installs the following tools:

1. npm
2. npx

Install React library using Node

Make a `package.json` file and add the dependencies (runtime) and devDependencies (build and type-checking tools):

```
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@types/react": "^18.2.0",
    "@types/react-dom": "^18.2.0",
    "esbuild": "^0.28.2",
    "typescript": "^5.0.0"
  },
```

What Changed from the TypeScript Example?

	TypeScript example (3-2)	React example (4-2)
Install `tsc`	`npm install -g typescript` (global)	`npm install` (local, in `node_modules`)
Where is `tsc`?	in your `PATH`	in `node_modules/.bin` (not in your `PATH`)
Role of `tsc`	**compile**: writes `.js` files to `outDir`	**type check only** (`noEmit`)
Who makes the JavaScript?	`tsc`	`esbuild` (it also bundles React)

Build using `esbuild`

We need to generate a single JavaScript file from the TypeScript source file.

```
npx esbuild src/App.tsx \
    --bundle \
    --outfile=app.js \
    --format=iife \
    --global-name=AppModule
```

`esbuild` is installed locally in `node_modules/.bin`, which is not in your shell's `PATH`. Running plain `esbuild` gives `command not found`, so we use `npx` to run the local copy.

esbuild automatically:

- Removes the TypeScript types and converts the code to JavaScript (it is **not** `tsc` and does **not** type-check)
- Finds all your imports (`import { useState } from 'react'`)
- Reads React from node_modules (installed by `npm install`) and includes it
- Bundles everything into app.js
- Creates the global `AppModule` variable

Two Ways to Run the Same Tool

`esbuild` lives in `node_modules/.bin`, which is not in your `PATH`.

	In the terminal	In `package.json` scripts
Command	`npx esbuild ...`	`esbuild ...`
Who finds `esbuild`?	`npx` looks in `node_modules/.bin`	`npm run` adds `node_modules/.bin` to `PATH`
Script file (no type check)	`run-npx.sh`	`run-npm-nocheck.sh` (calls `npm run build:nocheck`)

tsconfig.json for React

```
{
  "compilerOptions": {
    "target": "ES2018",
    "lib": ["DOM", "ES2018"],
    "jsx": "react-jsx",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "isolatedModules": true,
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "noEmit": true,
    "skipLibCheck": true
  },
  "include": ["src"]
}
```

Compare this with the `tsconfig.json` of the TypeScript example (3-2):

- **No `outDir` / `rootDir`, and `noEmit: true`:** `tsc` writes nothing. `esbuild` writes `app.js`.

Here, esbuild builds the JavaScript, while tsc checks the types.

- `"jsx": "react-jsx"`: the modern JSX transform. JSX becomes `jsx()` calls, so we do not need `import React`. Both `tsc` and `esbuild` read this setting.
- `"moduleResolution": "Bundler"`: imports such as `'react'` are resolved the way a bundler does (from `node_modules`).
- `isolatedModules`: every file must be convertible on its own, because `esbuild` converts one file at a time.
- `skipLibCheck`: do not type-check the `.d.ts` files in `node_modules`.

Add a Type Check to the Build

esbuild does not type-check, so we add the TypeScript compiler (`tsc`) as a separate step.

- `npm run check` runs `tsc --noEmit`: it only checks types and writes no files (`tsconfig.json` sets `noEmit`).
- `npm run build:nocheck` only bundles with esbuild.
- `npm run build` checks first, then bundles. If the check fails, nothing is built.
- We choose the global name "AppModule".

```
"scripts": {
  "check": "tsc --noEmit",
  "build": "npm run check && npm run build:nocheck",
  "build:nocheck": "esbuild src/App.tsx
      --bundle --outfile=app.js
      --format=iife
      --global-name=AppModule"
}
```

We can add `serve` command to run web server:

```
"serve": "python3 -m http.server 8000
  || python -m SimpleHTTPServer 8000
  || npx http-server -p 8000"
```

We have four scripts. Each one runs `npm install` first:

	No type check	With type check
`npx`	`run-npx.sh`	`run-npx-check.sh` (`npx tsc --noEmit`, then `npx esbuild`)
`npm run`	`run-npm-nocheck.sh` (`build:nocheck`)	`run.sh` (`build`)

Run the React Web Application

1. Open `src/index.html` in the web browser through the `file://` protocol (double-click the file). This works because `app.js` is a plain script.
2. Or run a local web server and open `http://localhost:8000/src/index.html`. This is closer to how a real website is served, and some browser features behave differently under `file://`.

```
> npm run serve
```

app.js and AppModule

The generated app.js is a huge JavaScript file (about 1 MB) because it contains React and ReactDOM too (the development build).

- It defines a global variable `AppModule` that holds what `App.tsx` exports (here, `mount`).

```
var AppModule = (() => {
  var __create = Object.create;
  var __defProp = Object.defineProperty;
  var __getOwnPropDesc = Object.getOwnPropertyDescriptor;
  var __getOwnPropNames = Object.getOwnPropertyNames;
  var __getProtoOf = Object.getPrototypeOf;
  var __hasOwnProp = Object.prototype.hasOwnProperty;
  ...
```

Connecting HTML and app.js

HTML

In the HTML file, there is a placeholder with an id "root".

```
<body>
  <div id="root"></div>
  ...
</body>
```

The app.js is imported.

```
  <script src="../app.js"></script>
```

Then, the `mount` function of `AppModule` connects the React components to the HTML's root placeholder.

```
<script>
  if (typeof AppModule !== 'undefined' && AppModule.mount) {
    AppModule.mount('root');
  } else {
    document.getElementById('root').innerHTML = `
      <div style="padding: 40px; text-align: center; color: #e74c3c;">
        <h2>Error: React app not found</h2>
        <p>Please run:</p>
        <p><code>npm install</code></p>
        <p><code>npm run build</code></p>
      </div>
    `;
  }
</script>
```

The App.tsx

The TypeScript + React file has the following structure:

```
import { useState, useEffect, type FormEvent } from 'react';
import { createRoot } from 'react-dom/client';

// Types and constants
interface Grade {...}

// Pure functions (no React)
function calculateFinalGrade(...) {...}

// Components
function GradeInput(...) {...}

// Main Component
function GradeCalculatorApp() {...}

// The mount function
export function mount(elementId: string) {...}
```

```
import { useState, useEffect, type FormEvent }
```

- `FormEvent` is a TypeScript type that describes the event object passed to a form handler such as `onSubmit`.
- TypeScript uses it to know that the event has methods like `e.preventDefault()`.
- The `type` keyword in `type FormEvent` means this import is used only for type checking.
- It is removed when the code is compiled, so it adds nothing to `app.js`.

Providing the mount function

In the `App.tsx` code, we export a `mount` function to be used in the JavaScript section of the HTML.

```
// Mount function to render the app
export function mount(elementId: string): void {
  const container = document.getElementById(elementId);
  if (!container) {
    throw new Error(`#${elementId} was not found`);
  }
  createRoot(container).render(<GradeCalculatorApp />);
}
```

- `mount('root')` finds the element with `id="root"` and renders `GradeCalculatorApp` inside it.
- `export` makes it available as `AppModule.mount`.

React Style: Three Sections

Most TypeScript + React functions have three sections:

1. Interface to define the Props type.
2. Business logic for computation or generating new data
3. Return JSX code

```
// 1. Interface
interface I { f: (T1) => T2 }
function f({f}: I) {
  // 2. Business Logic
  ...
  // 3. Return JSX
  return (
    <tag> ... </tag>
  );
}
```

Next: Read App.tsx

We know how to build and run the app. Next, we read `App.tsx` from top to bottom in three parts:

1. **4-3 App.tsx**: types, pure functions, and the parent component `GradeCalculatorApp` (the state)
2. **4-4 components**: `GradeInput`, `GradeList`, `GradeItem`, `GradeResult`
3. **4-5 review**: persistence with `localStorage` (extra), break the code on purpose, and judge it